

1. CONTENIDO.

Responda las siguientes preguntas:

1- ¿Qué ventajas ofrece Java 21 respecto a versiones anteriores y cómo impactan en el desarrollo con Spring Boot?

Una de las principales diferencias que tiene un impacto profundo en springboot es la posibilidad de manejar hilos de ejecución ligeros, a diferencia de los hilos tradicionales que tiene un alto costo en recursos y rendimiento los hilos virtuales son administrados por la maquina virtual de java y permiten manejar una gran cantidad de peticiones sin grandes costos, esto es un plus importante para las solicitudes que se hagan a cualquier api creada en spring pues cada petición es un hilo de ejecución mas ligero.

2- ¿Qué es la inyección de dependencias en Spring Boot y cómo se implementa?

Es un patrón de diseño que permite desacoplar los componentes de un software al permitir que una clase u objeto reciba las dependencias de forma externa en lugar de crearlas internamente, esto facilita la modularidad y el mantenimiento; comúnmente suele usarse junto a la inversión de dependencias como una buena practica.

En Java se pueden usar etiquetas para la inyección en lugar de hacerlo manualmente, las clases inyectables se pueden marcar como `@Service` `@Repository` `@Component` y se inyectan mediante la instrucción `@Autowired`

3- ¿Cuál es la diferencia entre una proyección y un DTO en Spring Data JPA? ¿Cuándo usar cada uno?

En una proyección se definen solo los atributos que se quieren cargar desde la base de datos creando una interfaz que define estos campos (los cuales pueden tener manipulaciones simples) es útil para optimizar consultas realizadas a la base de datos.

El repositorio de Spring data entonces puede obtener estos datos según dicte la interfaz.

Por otro lado un DTO es una clase “normal” para transferir datos entre las capas del software o definir la estructura para exportar o importar datos desde otras piezas de software como clientes Web, si es apropiado se puede definir como serializable para mejorar la comunicación. La idea es transferir solo la información necesaria sin exponer la entidad completa de la base de datos por ejemplo.

4- ¿Cuál es la principal novedad de Angular(desde la 2 en adelante) respecto a su antecesora AngularJs?

La diferencia es que el angular moderno posee una arquitectura basada en componentes, que mejora la modularidad de las aplicaciones construidas sobre el mismo.

5- ¿Qué diferencias existen entre los formularios reactivos y los formularios template-driven en Angular? ¿Cuál recomendarías para una aplicación empresarial?

Los formulario template-driven se basan en HTML y son fáciles de usar, actualmente lo ideal es usar formularios reactivos que a pesar de ser mas complejos y robustos permiten realizar validaciones y testing de forma apropiada, esto gracias que están basados en TypeScript que les da mayor funcionalidad.

6- ¿Qué es el ciclo de vida de un componente en Angular y cómo se puede aprovechar para mejorar el rendimiento o la lógica de la aplicación?

El ciclo de vida de los componentes en angular es la ruta que siguen los componentes desde que se crean hasta que se destruyen.

Esta ruta esta definida en fases las cuales se pueden manipular en código con métodos reservados como `ngOnInit()` `onOnChange()`.

Es importante conocer este ciclo de vida pues puede evitar errores al cargar datos en el momento apropiado, también se puede mejorar el rendimiento evitando crear lógica pesada en métodos que se ejecuten repetidamente durante el ciclo de vida

7- ¿Qué son las directivas en Angular y mencione algunas?

Son comandos o instrucciones que se pueden dar desde el apartado HTML del componente angular y permiten transformar el DOM, esto permite darle al HTML funcionalidades propias de un lenguaje de programación (Aunque HTML no lo sea), como aplicar condicionales `ngIf` y crear ciclos como `ngFor` entre otros.